Université du Québec à Chicoutimi

Département d'informatique et de mathématique

Rapport de Sprint 1

Cours : 8INF887 – Apprentissage profond

Hiver 2026 – Groupe 01

Projet :

Sous-titrage et traduction automatique de films/vidéos

Étudiant :

BEHAREL Armand

(Travail individuel)

Professeur : Kévin Bouchard, Ph.D.

Date de remise : 27 mars 2026

Table des matières

1 Travail réalisé **2**
1.1 1.1 — Base de données : Mozilla Common Voice (FR) 2
1.2 1.2 — Tokenizer BPE 3
1.3 1.3 — Dataset PyTorch (Encoder-Décodeur) 4
1.4 1.4 — Architecture Transformer Encoder-Décodeur 4
1.5 1.5 — Entraînement : premiers tests 5

2 Bilan du Sprint 1 **6**
2.1 Objectifs atteints 6
2.2 Difficultés rencontrées 7
2.3 Réflexion sur l'architecture Transformer 7

3 Planification du Sprint 2 **7**
3.1 Objectif principal : passage au Conformer 7

Travail réalisé

Ce sprint couvre la mise en place complète de la pipeline de préparation des données, du tokenizer, de l'architecture Transformer encoder-décodeur, et des premiers tests d'entraînement.

1.1 — Base de données : Mozilla Common Voice (FR)

Base de données choisie

Mozilla Common Voice 17.0 – Français (téléchargé sur un Hugging Face qui n'existe plus)
Source : `https://datacollective.mozillafoundation.org/organization/cmfh0j9o10006ns07jq45h7xk`
Langue cible : français (avec marqué si accent québécois)

Le jeu de données Mozilla Common Voice a été retenu pour les raisons suivantes :

- **Volume** : plus de 1600 heures de parole validée en français
- **Qualité** : annotations de locuteurs bénévoles, validées par la communauté
- **Diversité** : accents variés, incluant des locuteurs québécois

Structure du jeu de données sur disque :

```
data/common_voice/
          audio/fr/
                train/  fr_train_0.tar ... fr_train_14.tar
                dev/    fr_dev_0.tar
                test/   fr_test_0.tar
          transcript/fr/
              validated.tsv
              train.tsv
              test.tsv
              dev.tsv
```

Listing 1 – Arborescence des fichiers Common Voice

Le script `download_dataset.py` réalise les opérations suivantes :

1. Extraction automatique des archives `.tar`
2. Lecture des fichiers `.tsv` (métadonnées : transcription, accent, durée, genre)
3. Conversion MP3 → WAV 16 kHz mono avec `librosa` (On se limite à 150h de données car le temps de d'entraînement sera trop long)
4. Filtrage par durée (1 – 15 secondes)
5. Détection des clips québécois (mots-clés dans le champ `accent`)
6. Génération des manifests JSON (un objet par ligne)

Format d'un manifest JSON (une entrée) :

```
{
  "audio_filepath": "G:/Armand/New-Whisper/data/train/train_0000123
   .wav",
  "text": "bonjour comment allez-vous aujourd'hui",
```

```
  "duration": 3.142,
  "accent": "canadian",
  "is_qc": true,
  "age": "twenties",
  "gender": "male"
}
```

Listing 2 – Exemple d'entrée manifest

Statistiques obtenues (objectif : 150 h d'entraînement) :

Split	Clips	Durée totale
train	∼145 000	150 h
validation	∼15 000	15 h
test	∼15 000	15h

TABLE 1 – Statistiques approximatives du dataset préparé

1.2 — Tokenizer BPE

Tokenizer

SentencePiece BPE – vocabulaire de 1 000 tokens
Bibliothèque : `sentencepiece` (Google)

Un tokenizer *Byte Pair Encoding* (BPE) a été entraîné sur le corpus de transcriptions du jeu d'entraînement. Ce choix est motivé par :

- Sa capacité à gérer les mots hors-vocabulaire par sous-mots
- Son efficacité sur les langues comme le français
- Sa compatibilité native avec `sentencepiece`

Tokens spéciaux définis :

Token	ID	Rôle
`<pad>`	0	Rembourrage (padding)
`<unk>`	1	Token inconnu
`<s>`	2	Début de séquence (BOS)
`</s>`	3	Fin de séquence (EOS)
`<blank>`	–	Blank CTC (réservé)

TABLE 2 – Tokens spéciaux du tokenizer

Exemple de tokenisation :

```
# Entr e : "bonjour comment allez vous"
# Tokens : ['  bonjour  ', '  comment  ', '  allez  ', '  vous  ']
# IDs    : [45, 123, 67, 89]

# Avec teacher forcing :
tgt_in  = [<s>,  45,  123, 67,  89 ]  # BOS + tokens
```

```
tgt_out = [45,  123,  67,  89, </s>]  # tokens + EOS
```

Listing 3 – Exemple de tokenisation BPE

1.3 — Dataset PyTorch (Encoder-Décodeur)

Le module `dataset.py` implémente un `Dataset` PyTorch adapté à l'architecture encoder-décodeur avec *teacher forcing*.

Model :

	Encoder-Décodeur
Entrée encodeur	mel [T, 80]
Entrée décodeur	tokens décalés droite [U+1]
Cible	tokens décalés gauche [U+1]
Loss	CrossEntropyLoss

Table 3 – Comparaison CTC vs Encoder-Décodeur

Le *teacher forcing* consiste à fournir au décodeur le vrai token précédent pendant l'entraînement (plutôt que sa propre prédiction), ce qui accélère la convergence.

Pipeline de traitement audio :

1. Chargement WAV avec `soundfile`
2. Conversion mono si nécessaire
3. Rééchantillonnage à 16 kHz si nécessaire
4. Log-Mel Spectrogram : 80 bandes, FFT 512, hop 160 (10 ms/frame)
5. SpecAugment (entraînement) : masquage fréquences + temps
6. Normalisation par utterance (moyenne=0, écart-type=1)
7. Sortie : tenseur [T, 80]

1.4 — Architecture Transformer Encoder-Décodeur

J'ai d'abord exploré les approches les plus simples à mettre en œuvre, en commençant par les modèles CTC + Transformeur, faciles à entraîner mais encore éloignés de l'état de l'art. Je me suis ensuite intéressé aux architectures Seq2Seq encodeur-décodeur, qui offrent une modélisation plus riche de la séquence audio vers le texte. Dans la continuité, j'ai étudié les modèles RNN-T, adaptés à la reconnaissance vocale et capables de gérer l'alignement de manière implicite. Cela m'a naturellement conduit vers les Conformer, qui représentent aujourd'hui l'état de l'art en combinant efficacement attention et convolutions. Enfin, j'ai également examiné le modèle Moshi développé par le laboratoire Kyutai, financé par Free, mais celui-ci n'a pas été retenu car il est principalement pensé pour du dialogue vocal en temps réel plutôt que pour une tâche de transcription classique.

Architecture globale :

— **Encodeur audio** :

  - Projection du spectrogram via convolutions 1D
  - Encodage positionnel sinusoïdal
  - N couches Transformer
- **Décodeur textuel** :
  - Embedding des tokens
  - Encodage positionnel appris
  - M couches Transformer décodeur (self-attention masquée + cross-attention)
- **Tête de prédiction** : couche linéaire → vocabulaire (1 000 tokens)

Hyperparamètres du modèle (configuration testée) :

Paramètre	Valeur
Dimension du modèle (d_{model})	256
Nombre de têtes d'attention	4
Couches encodeur	4
Couches décodeur	4
Dimension FFN	1024
Dropout	0.1
Taille du vocabulaire	1 000
Paramètres totaux	∼15 M

TABLE 4 – Hyperparamètres de l'architecture Transformer

1.5 — Entraînement : premiers tests

Configuration d'entraînement :

Paramètre	Valeur
Optimiseur	AdamW
Learning rate	3×10^{-4} avec warmup
Scheduler	CosineAnnealingLR
Batch size	8
Gradient clipping	1.0
Loss	CrossEntropyLoss (ignore padding)
Curriculum learning	Tri par durée croissante

TABLE 5 – Configuration d'entraînement

Observations des premiers tests :

- La loss d'entraînement descend correctement sur les premiers batches, confirmant que le modèle apprend bien
- Le curriculum learning (courts exemples en premier) facilite la stabilité au début de l'entraînement
- La validation par WER (*Word Error Rate*) sera mise en place complètement lors du sprint suivant

— Le SpecAugment améliore la robustesse sans ralentir significativement l'entraînement

— Premier test encourangeant, il devine très régulièrement le premier mot correctement (Il, La, Le, etc.)

```
============================================================
REF : ils seraient également affectueux, intelligents et auraient une forte personnalité.
HYP : la parole est à monsieur bourg, pour soutenir l'amendement numéro cent quatre-vingt-dix-sept, pour soutenir l'amen
dement numéro cent soixante-dix-dix.

============================================================
REF : la fête patronale a lieu pour la saint laurent, agrémentée par bals, feu d'artifice.
HYP : la parole est à monsieur jean-louis, pour soutenir l'amendement numéro cent quatre-vingt-dix-sept.

============================================================
REF : il pouvait exécuter quatre instructions par cycle d'horloge.
HYP : il est le fils de l'université de l'architecture de l'architecture de l'institutte.
```

FIGURE 1 – Test du model en cours de développement

Pipeline complète opérationnelle :

```python
# 1. Pr paration des donn es
python download_dataset.py

# 2. Entra nement du tokenizer
python train_tokenizer.py

# 3. V rification du dataset
python dataset.py

# 4. Entra nement du mod le
python train.py
```

Listing 4 – Ordre d'exécution des scripts

Bilan du Sprint 1

Objectifs atteints

- ✓ Mise en place de la pipeline de données Common Voice (extraction, conversion, filtrage)
- ✓ Génération des manifests JSON pour train/validation/test
- ✓ Entraînement du tokenizer BPE français (1 000 tokens)
- ✓ Implémentation du `Dataset` PyTorch avec teacher forcing
- ✓ Architecture Transformer encoder-décodeur
- ✓ Boucle d'entraînement fonctionnelle avec premiers tests
- ✓ SpecAugment pour la robustesse
- ✓ Curriculum learning (tri par durée)

Difficultés rencontrées

- **Extraction des archives** : les 15 fichiers `.tar` d'entraînement représentent un volume conséquent (∼7-9 heures d'extraction)
- **Indexation des MP3** : la structure imbriquée des archives nécessitait une indexation récursive par nom de fichier (stem)
- **Compatibilité audio sur Windows** : utilisation de `librosa` plutôt que `torchaudio` pour la lecture des MP3 (à permis de doubler la vitesse d'extraction)
- **Alignment teacher forcing** : vérification rigoureuse que `tgt_in[1:] == tgt_out[:-1]` pour tous les samples

Réflexion sur l'architecture Transformer

Après avoir implémenté et testé l'architecture Transformer standard, plusieurs limitations ont été identifiées pour la reconnaissance vocale :

- Le Transformer traite les séquences sans biais inductif vers la *localité temporelle*
- Les convolutions locales capturent mieux les motifs acoustiques courts (phonèmes)
- Le Transformer pur peut être plus difficile à entraîner sur de petits datasets audio

Ces observations motivent le passage au **Conformer** pour le sprint 2.

Planification du Sprint 2

Objectif principal : passage au Conformer

Le Sprint 2 sera entièrement consacré à la migration de l'encodeur Transformer standard vers une architecture **Conformer**, qui combine convolutions et attention pour mieux capturer les dépendances locales et globales dans le signal audio.

Conformer vs Transformer

Conformer (Gulati et al., 2020) = Transformer + Convolution
Chaque bloc Conformer :

1. Feed-Forward (demi-step)
2. Multi-Head Self-Attention
3. **Convolution Module** (depthwise separable conv)
4. Feed-Forward (demi-step)
5. Layer Norm

Avantage : capture les dépendances locales (conv) et globales (attention)

Tâches planifiées pour le Sprint 2 :

1. **Implémentation du bloc Conformer**

- Module de convolution depthwise separable
- Intégration dans la boucle encoder-décodeur existante

2. **Décodage par beam search**
   - Implémentation du décodage autorégressif avec beam search
   - Remplacement du décodage greedy utilisé en sprint 1
3. **Métriques et logging**
   - WER et CER (Character Error Rate)
   - Intégration d'un logger (`wandb` ou `tensorboard`)
4. **Inférence sur une vidéo**
   - Pipeline d'extraction audio depuis une vidéo (`ffmpeg`)
   - Génération de sous-titres au format SRT

Calendrier prévisionnel Sprint 2 :

Semaine	**Tâche**
28 mars – 3 avril	Implémentation du Conformer + intégration
4 – 17 avril	Entraînement de models
8 – 10 avril	Rapport sprint 2
17 – 22 avril	Benchmarking, beam search, WER
22 avril – 3 mai	Pipeline inférence vidéo + rapport final
Remise	**Dimanche 3 mai 2026**

Table 6 – Calendrier du Sprint 2